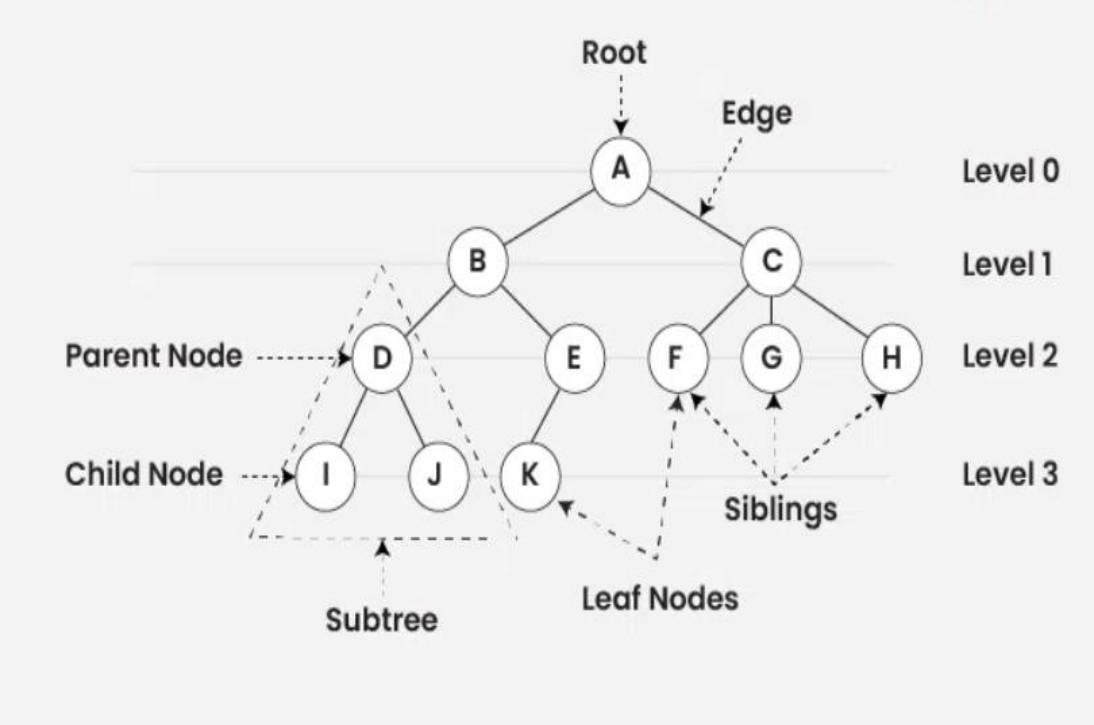
OBJECT ORIENTED PROGRAMMING USING C++

Study Materials

Tree:

- Tree data structure is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search.
- It is a collection of nodes that are connected by edges and has a hierarchical relationship between the nodes.
- The topmost node of the tree is called the root, and the nodes below it are called the child nodes.
- Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.

Representation of Trees:

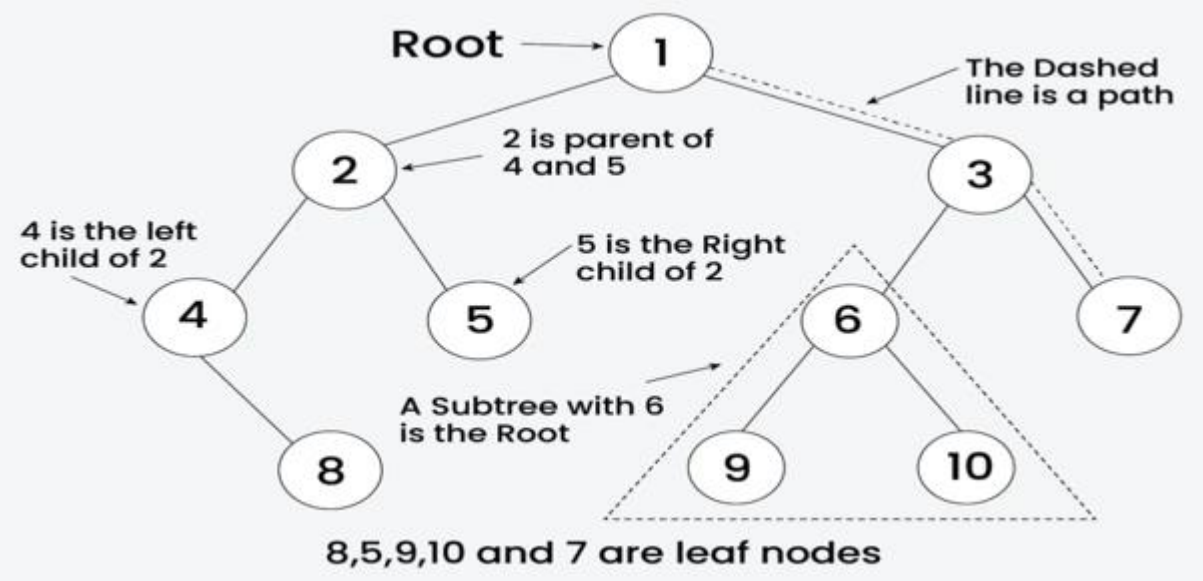


Types of Tree:

- Tree data structure can be classified into three types based upon the number of children each node of the tree can have. The types are:
- Binary tree: In a binary tree, each node can have a maximum of two children linked to it. Some common types of binary trees include full binary trees, complete binary trees, balanced binary trees, and degenerate or pathological binary trees. Examples of Binary Tree are Binary Search Tree and Binary Heap.
- Ternary Tree: A Ternary Tree is a tree data structure in which each node has at most three child nodes, usually distinguished as “left”, “mid” and “right”.
- N-ary Tree or Generic Tree: Generic trees are a collection of nodes where each node is a data structure that consists of records and a list of references to its children(duplicate references are not allowed). Unlike the linked list, each node stores the address of multiple nodes.

Binary Tree:

- Binary Tree is a non-linear and hierarchical data structure where each node has at most two children referred to as the left child and the right child.
- The topmost node in a binary tree is called the root, and the bottom-most nodes are called leaves.

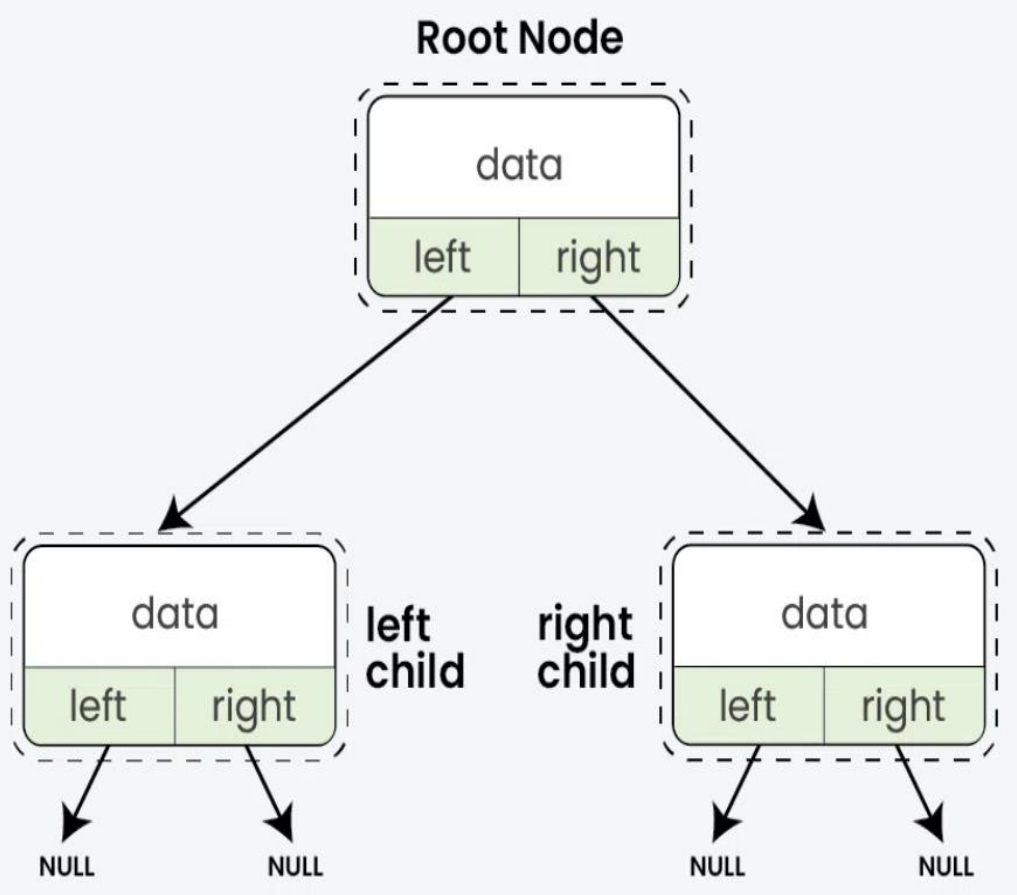


Binary tree:

- A Binary Tree is a tree data structure where each node has at most two children, referred to as the left child and the right child.
- **Key Points:**
- Root node: The topmost node in the tree.
- Leaf node: A node that has no children.
- Sub-trees: The left and right subtreesof any node.
- **Real-life Examples:**
- Hierarchical organization structures.
- Decision-making processes (e.g., binary search).

Representation of Binary Tree:

- Each node in a Binary Tree has three parts:
- Data
- Pointer to the left child
- Pointer to the right child



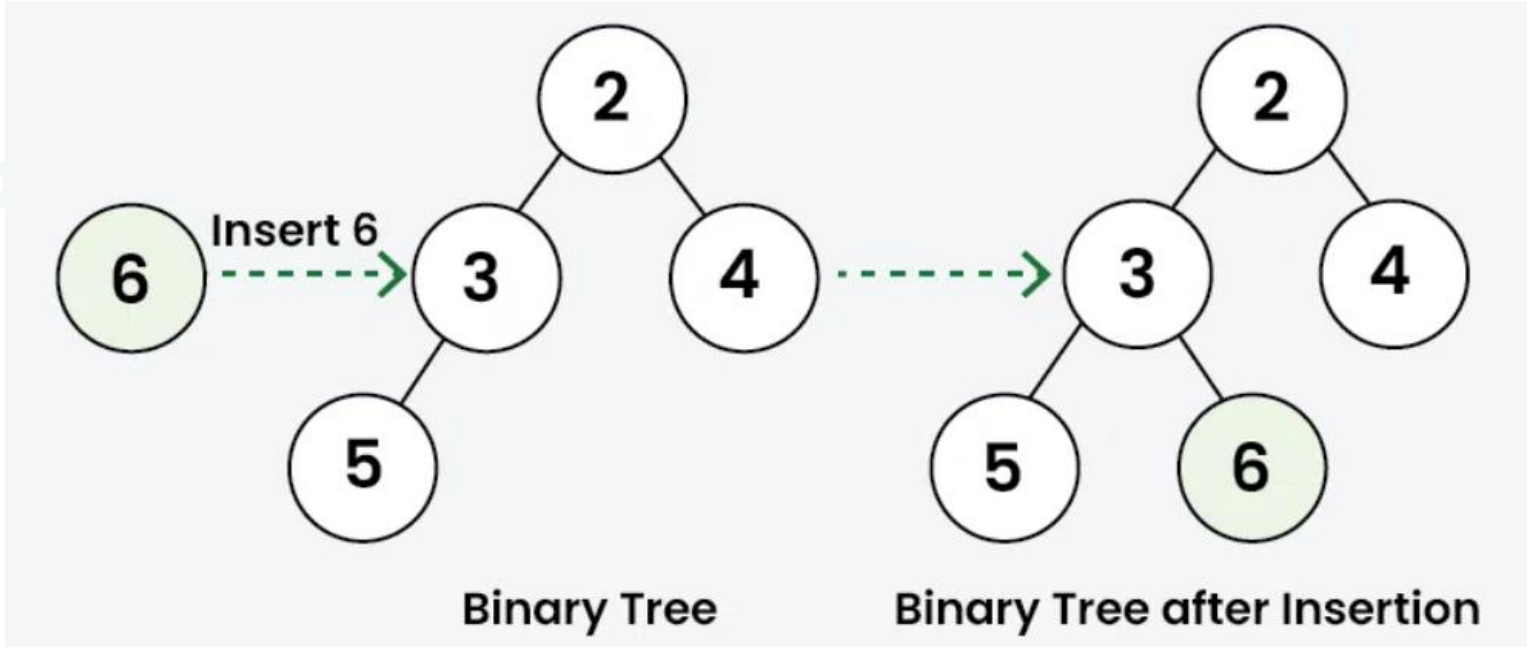
Binary Tree Operations:

Operation	Time Complexity	Auxiliary Space
In-Order Traversal	O(n)	O(n)
Pre-Order Traversal	O(n)	O(n)
Post-Order Traversal	O(n)	O(n)
Insertion (Unbalanced)	O(n)	O(n)
Searching (Unbalanced)	O(n)	O(n)
Deletion (Unbalanced)	O(n)	O(n)

Insertion in Binary Tree:

- Inserting elements means add a new node into the binary tree.
- As we know that there is no such ordering of elements in the binary tree, So we do not have to worry about the ordering of node in the binary tree.
- We would first creates a root node in case of empty tree.
- Then subsequent insertions involve iteratively searching for an empty place at each level of the tree.
- When an empty left or right child is found then new node is inserted there.
- By convention, insertion always starts with the left child node.

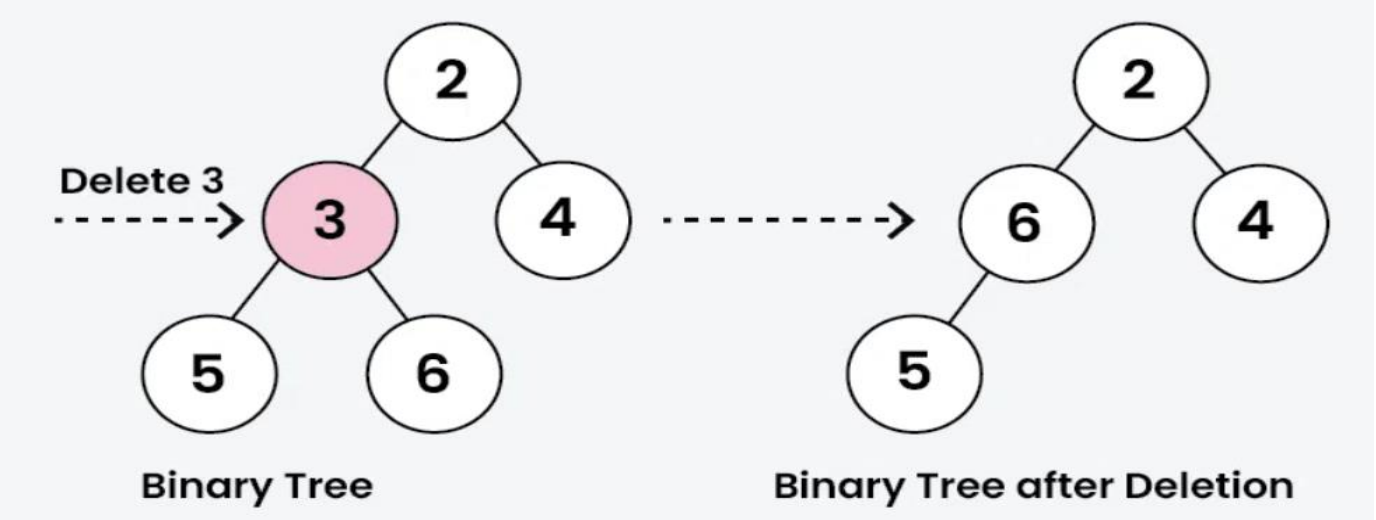
Insertion in Binary Tree:



Deletion in Binary Tree:

- Deleting a node from a binary tree means removing a specific node while keeping the tree's structure.
- First, we need to find the node that want to delete by traversing through the tree using any traversal method.
- Then replace the node's value with the value of the last node in the tree (found by traversing to the rightmost leaf), and then delete that last node.
- This way, the tree structure won't be effected. And remember to check for special cases, like trying to delete from an empty tree, to avoid any issues.

Deletion in Binary Tree:



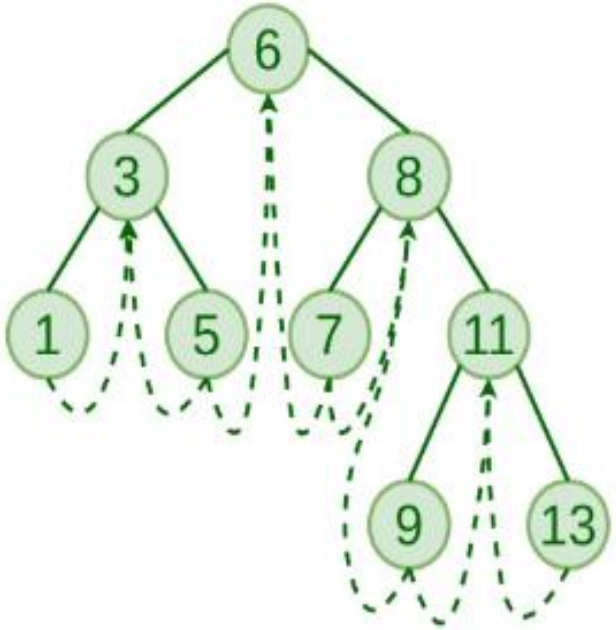
Threaded Binary Tree:

- A threaded binary tree is a type of binary tree data structure where the empty left and right child pointers in a binary tree are replaced with threads that link nodes directly to their in-order predecessor or successor, thereby providing a way to traverse the tree without using recursion or a stack.
- Threaded binary trees can be useful when space is a concern, as they can eliminate the need for a stack during traversal.
- However, they can be more complex to implement than standard binary trees.

Types of Threaded Binary Tree:

- There are two types of threaded binary trees.
- Single Threaded: Where a NULL right pointers is made to point to the in-order successor (if successor exists)
- Double Threaded: Where both left and right NULL pointers are made to point to in-order predecessor and in-order successor respectively.
- The predecessor threads are useful for reverse in-order traversal and post-order traversal.
- The threads are also useful for fast accessing ancestors of a node.

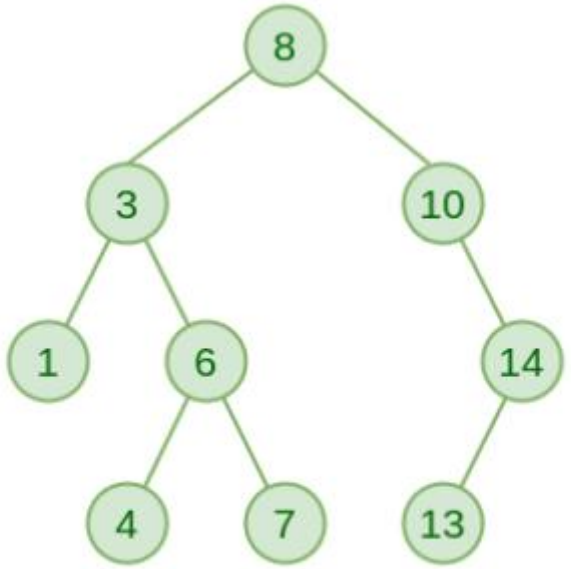
Double Threaded Binary Tree:



Binary Search tree:

- A binary search tree follows some order to arrange the elements.
- In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node.
- This rule is applied recursively to the left and right sub-trees of the root.
- The properties that separate a binary search tree from a regular binary tree is
- All nodes of left sub-tree are less than the root node
- All nodes of right sub-tree are more than the root node
- Both sub-trees of each node are also BSTs i.e. they have the above two properties

Binary Search tree:



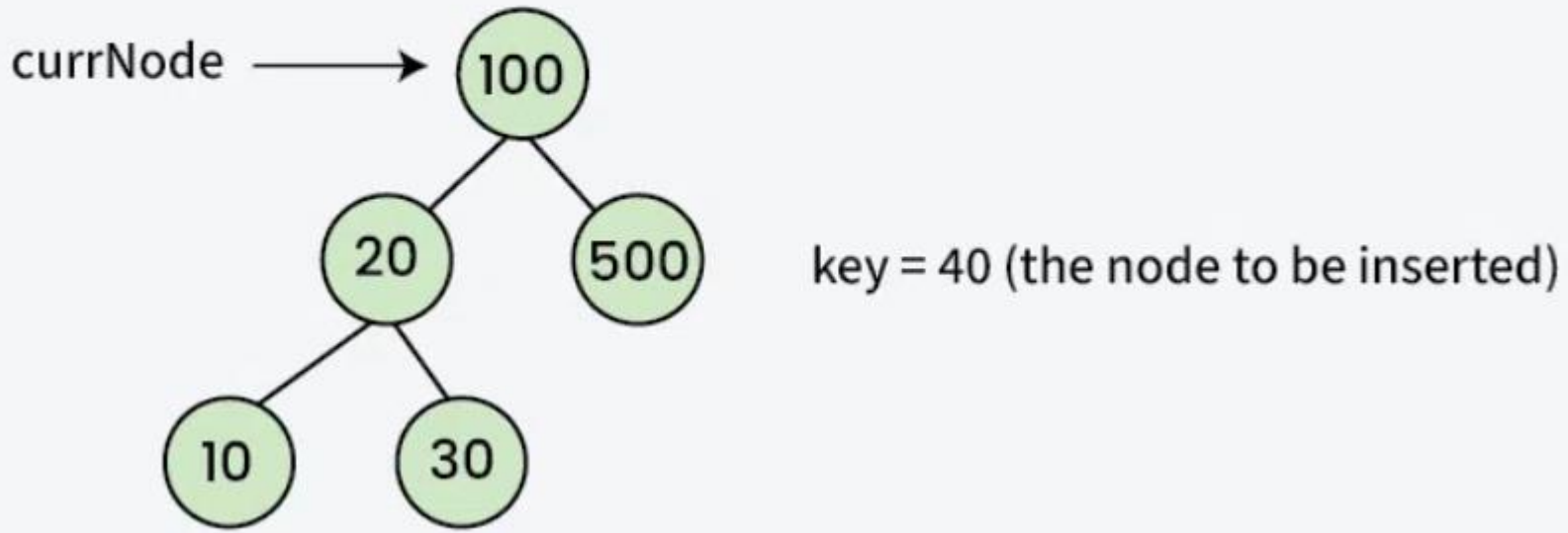
Binary Search Tree

Insertion in Binary Search Tree:

- A new key is always inserted at the leaf by maintaining the property of the binary search tree.
- We start searching for a key from the root until we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node. The below steps are followed while we try to insert a node into a binary search tree:
- Initilize the current node with root node
- Compare the key with the current node.
- Move left if the key is less than or equal to the current node value.
- Move right if the key is greater than current node value.
- Repeat steps 2 and 3 until you reach a leaf node.
- Attach the new key as a left or right child based on the comparison with the leaf node’s value.

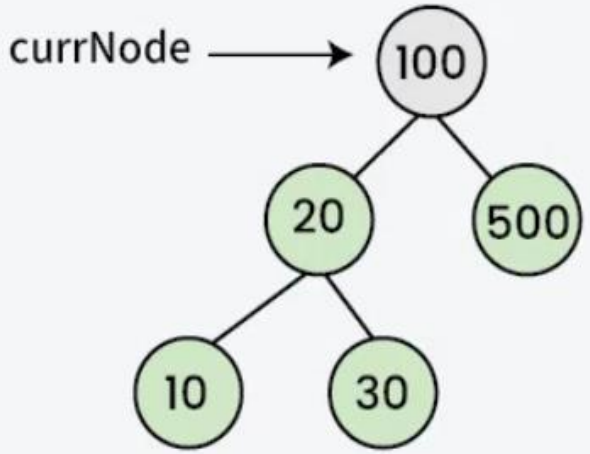
Insertion in Binary Search Tree:

01 Step | Consider the following BST



Insertion in Binary Search Tree:

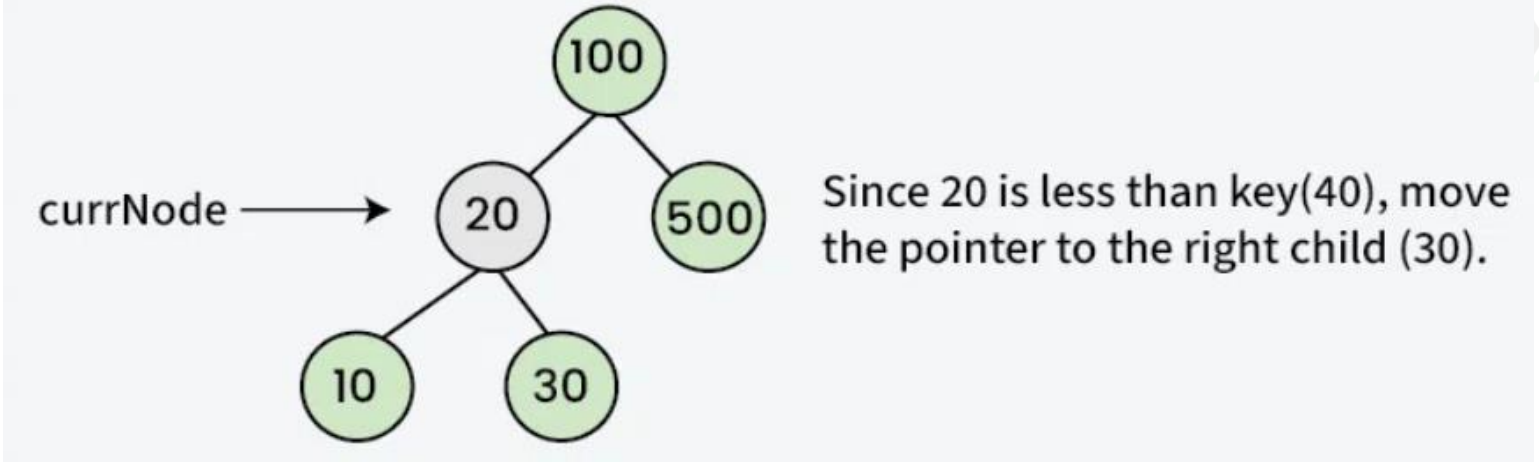
02 Step | Comparing key with root node



Since 100 is greater than key(40), move the pointer to the left child (20).

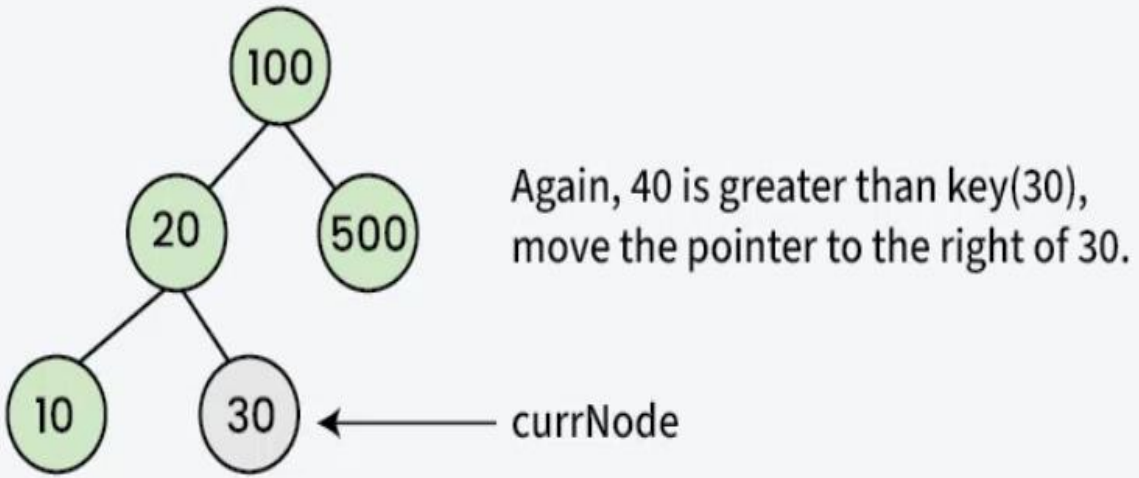
Insertion in Binary Search Tree:

03 Step | Comparing key with left child root node

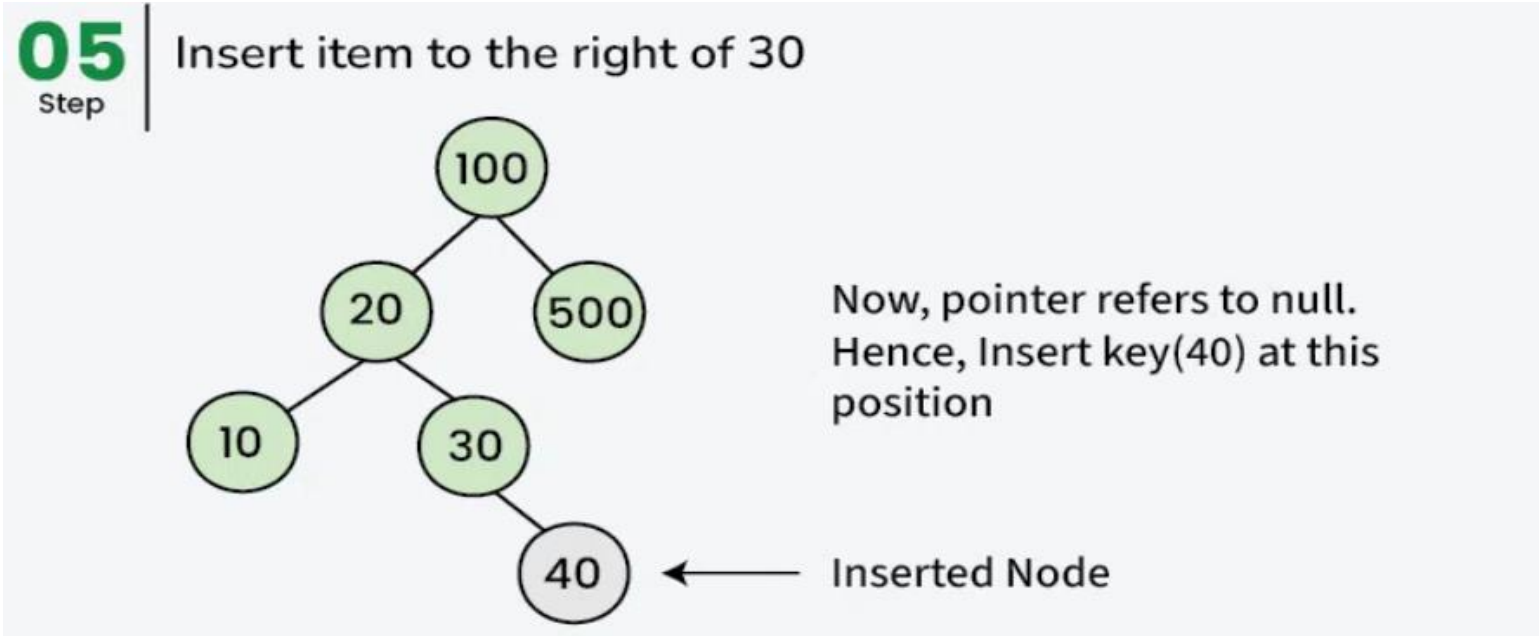


Insertion in Binary Search Tree:

04 Step | Comparing key with the right child of 20



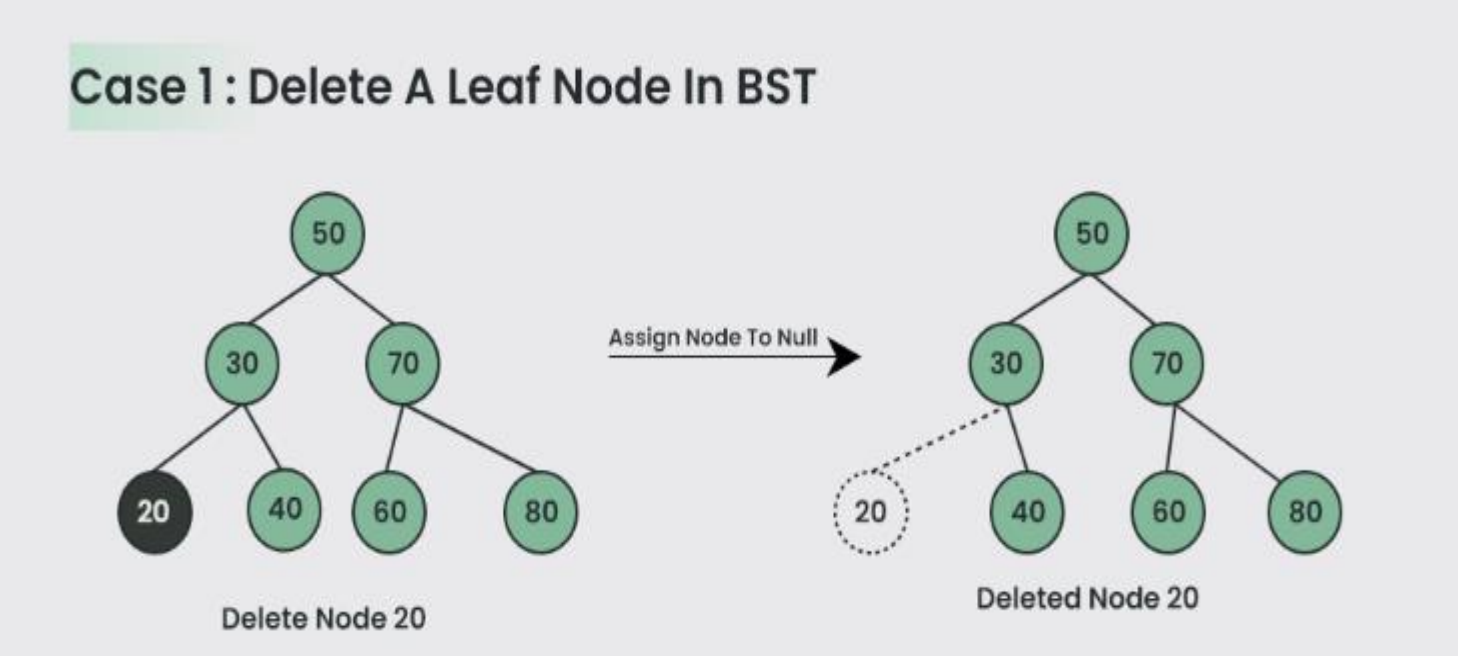
Insertion in Binary Search Tree:



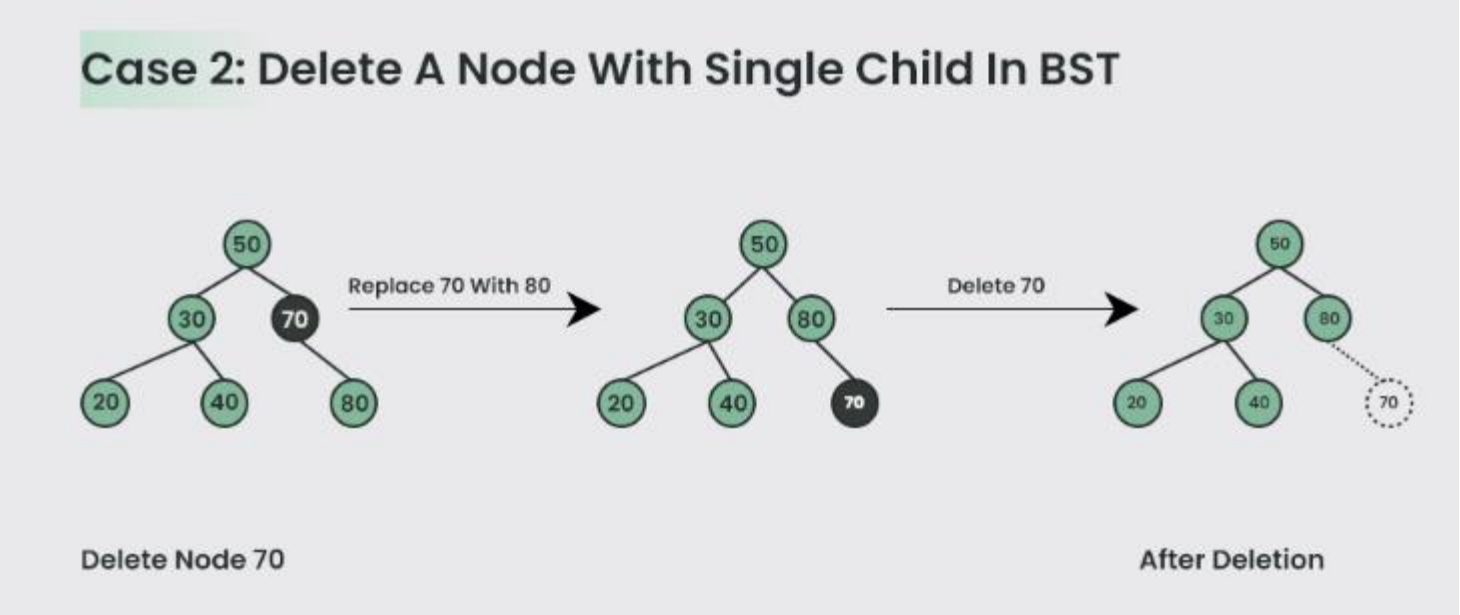
Deletion Operation:

- In a binary search tree, we must delete a node from the tree by keeping in mind that the property of BST is not violated.
- To delete a node from BST, there are three possible situations occur
- The node to be deleted is the leaf node, or,
- The node to be deleted has only one child, and,
- The node to be deleted has two children.

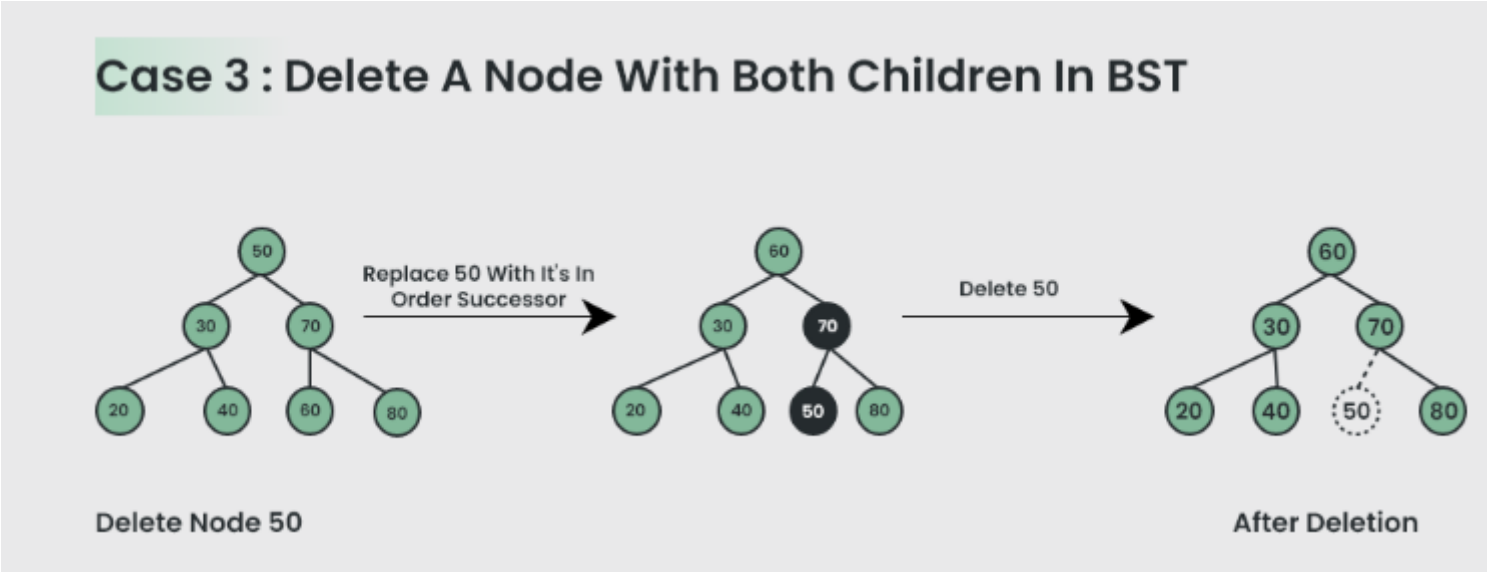
Deletion Operation:



Deletion Operation:



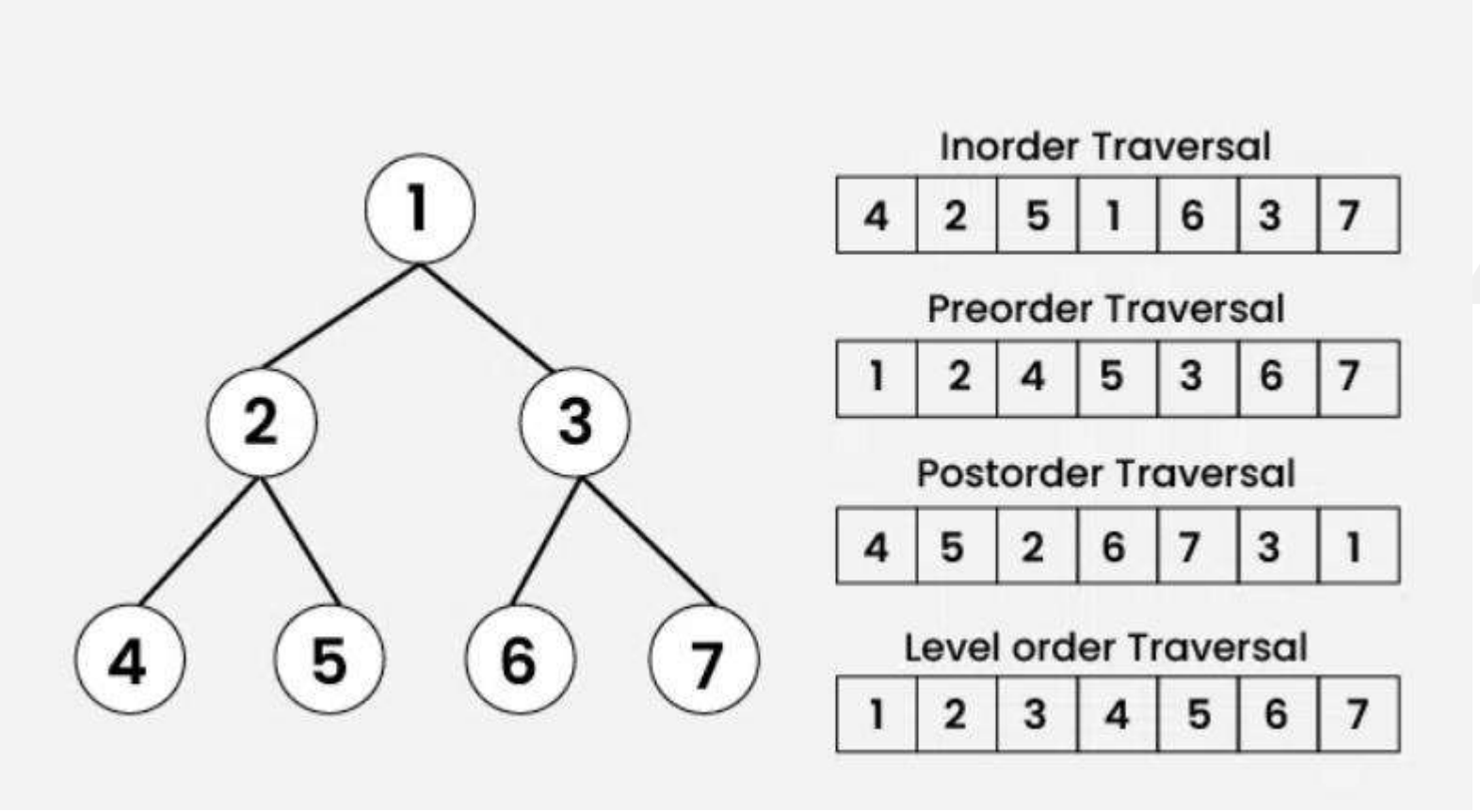
Deletion Operation:



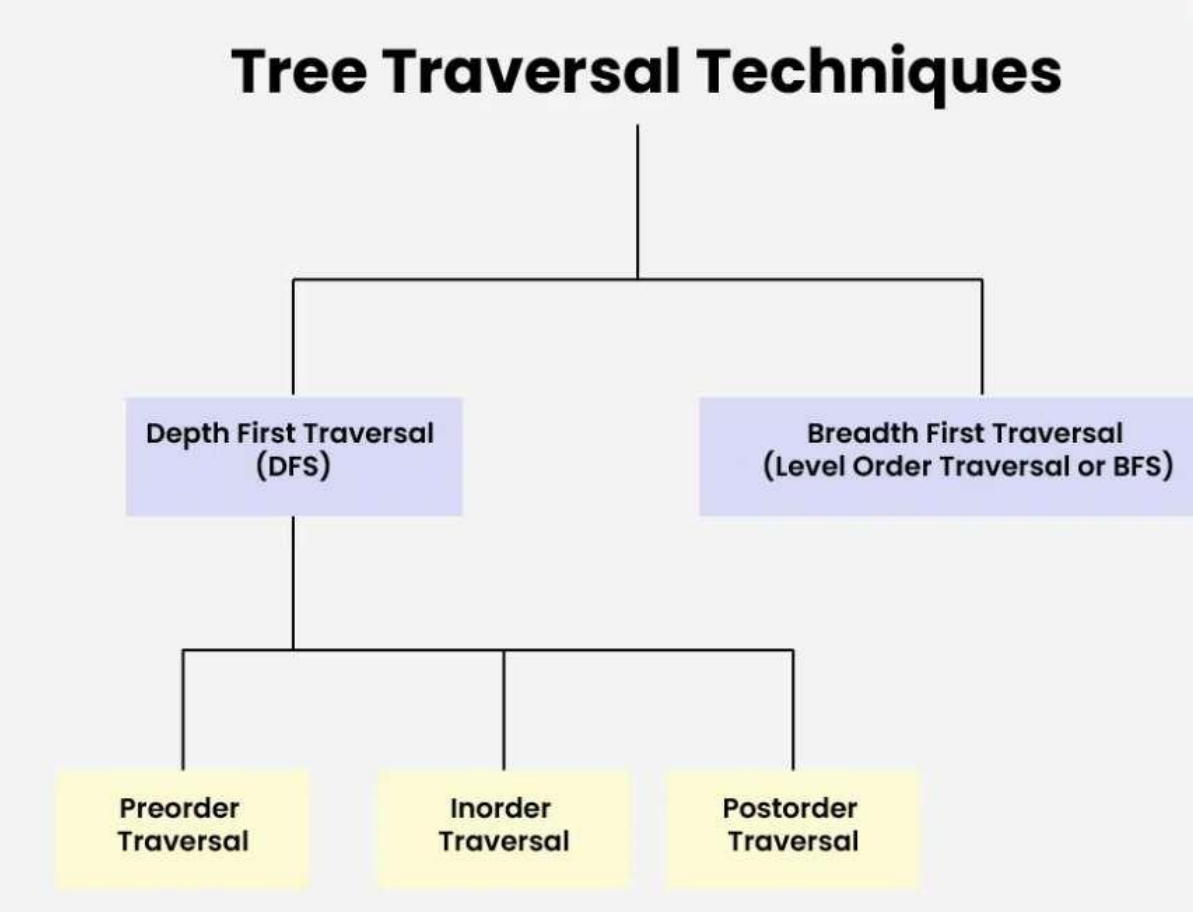
Tree traversal:

- The term 'tree traversal' means traversing or visiting each node of a tree.
- There is a single way to traverse the linear data structure such as linked list, queue, and stack.
- Whereas, there are multiple ways to traverse a tree that are listed as follows –
- Pre-order traversal
- In-order traversal
- Post-order traversal

Tree Traversal:

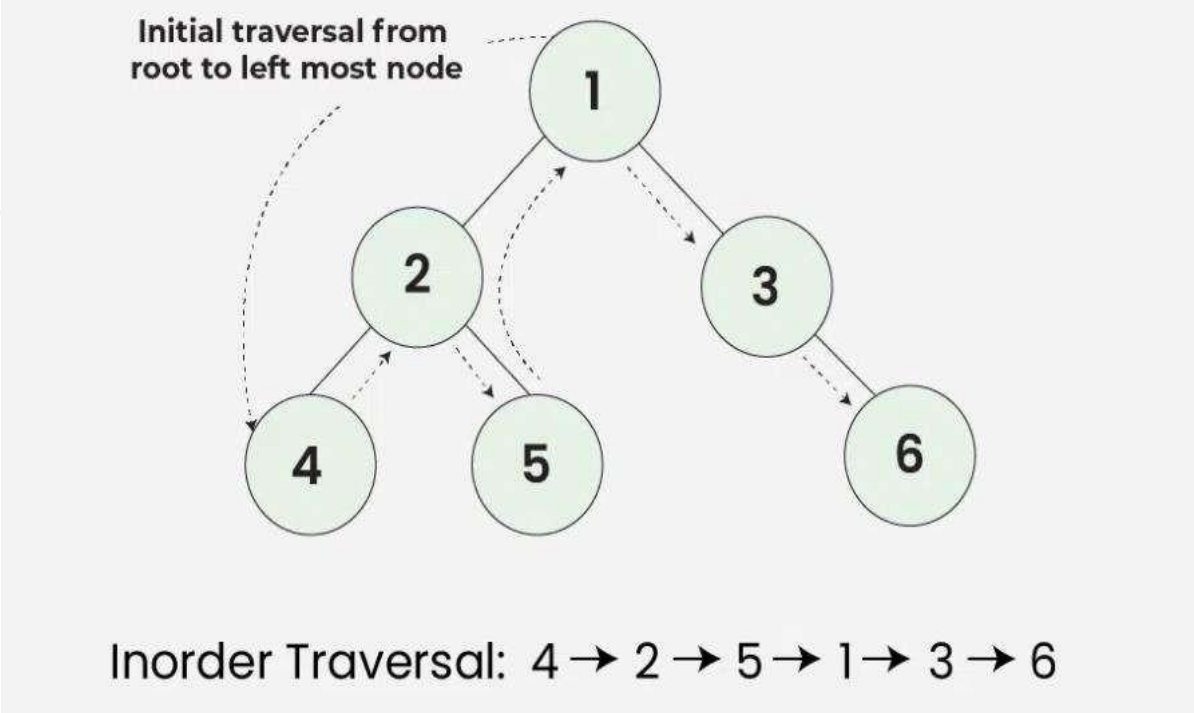


Tree Traversal:



In-order Traversal:

- In-order traversal visits the node in the order: Left -> Root -> Right

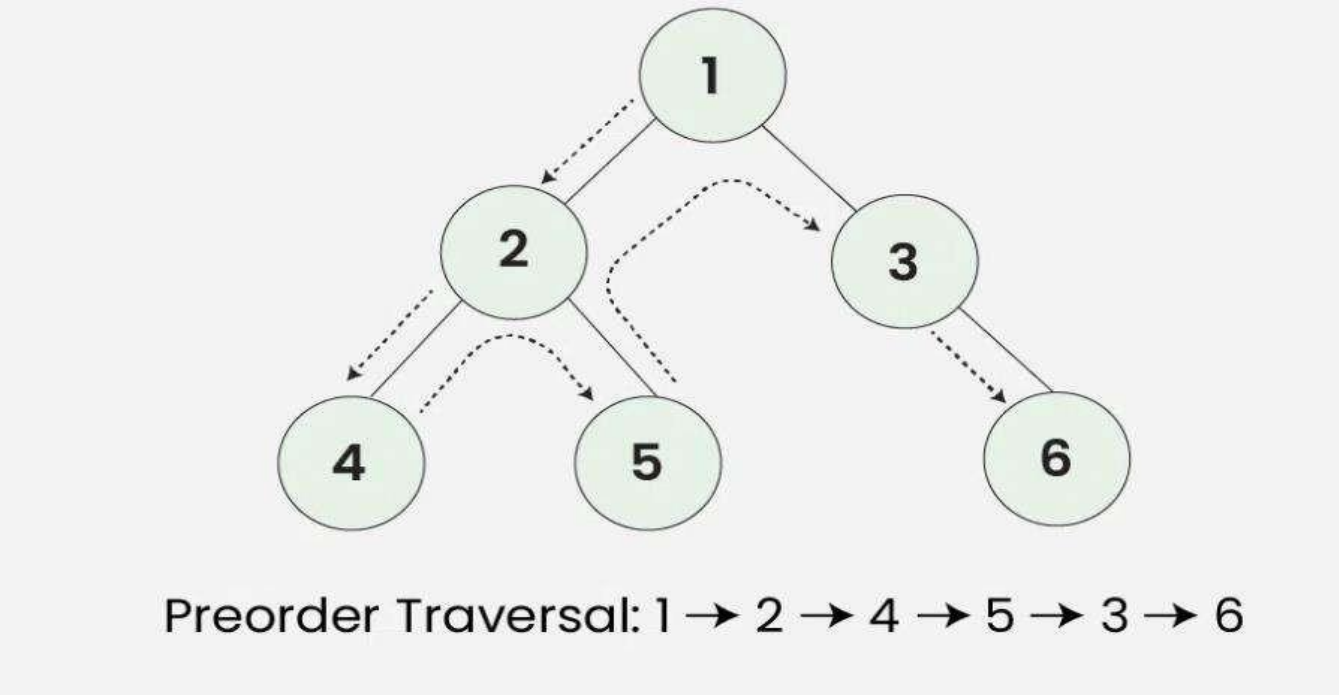


Algorithm for In-order Traversal:

- In-order(tree):
- Traverse the left sub-tree, call In-order(left->sub-tree)
- Visit the root.
- Traverse the right sub-tree, call In-order(right->sub-tree)
- **Uses of In-order Traversal:**
- In the case of binary search trees (BST), In-order traversal gives nodes in non-decreasing order.
- To get nodes of BST in non-increasing order, a variation of In-order traversal where In-order traversal is reversed can be used.
- In-order traversal can be used to evaluate arithmetic expressions stored in expression trees.

Pre-order traversal:

- Preorder traversal visits the node in the order: Root -> Left -> Right

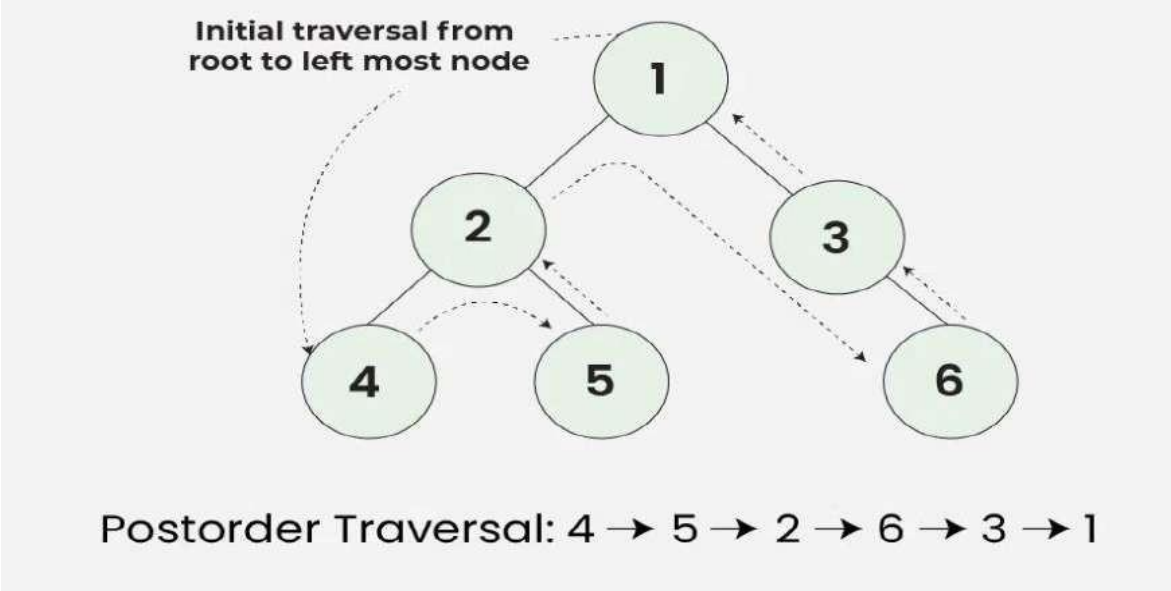


Algorithm for Preorder Traversal:

- Preorder(tree):
- Visit the root.
- Traverse the left sub-tree, call Preorder(left->subtree)
- Traverse the right sub-tree, call Preorder(right->subtree)
- **Uses of Preorder Traversal:**
- Preorder traversal is used to create a copy of the tree.
- Preorder traversal is also used to get prefix expressions on an expression tree.

Post-order traversal:

- Post- order traversal visits the node in the order: Left -> Right -> Root



Algorithm for Post-order Traversal:

- **Post-order(tree) :**
- Traverse the left sub-tree, call Post-order(left->sub-tree)
- Traverse the right sub-tree, call Post-order(right->sub-tree)
- Visit the root
- **Uses of Post-order Traversal:**
- Post-order traversal is used to delete the tree. See the question for the deletion of a tree for details.
- Post-order traversal is also useful to get the postfix expression of an expression tree.
- Post-order traversal can help in garbage collection algorithms, particularly in systems where manual memory management is used.